Desarrollar software a partir de la integración de sus módulos componentes. G8 A8 -220501096 - AA1 - EV01.

Nombre:

JULIAN ANDRES COLLAZOS CARDOSO

Instructor:

JHON FAIBER CERQUERA SANCHEZ

SERVICI0 NACIONAL DE APRENDIZAJE SENA

ANALISIS Y DESARROLLO SOFTWARE

FICHA:3282766

BOGOTA DC 2026

Introducción

Este documento presenta la evidencia de desarrollo e integración de módulos del Sistema de Gestión de Citas Médicas. Se elaboró tomando como referencia el proyecto entregado, cuya solución principal se encuentra en la carpeta “PaginaWebClinina” y está organizada en una aplicación web ASP.NET Web Forms con C#, una capa de entidades, una capa de lógica de negocio y una capa de acceso a datos. El proyecto también incluye un archivo de solución de Visual Studio y una exportación de base de datos SQL Server en formato BACPAC.

El propósito es describir la arquitectura, los módulos, la navegación, los requerimientos funcionales, las medidas de seguridad y las pruebas que deben realizarse para comprobar que los componentes funcionan de manera integrada. Las pruebas se presentan como plan de validación; no se afirma que hayan sido ejecutadas si no existe evidencia de ejecución.

Objetivo general

Documentar el desarrollo e integración de los módulos del Sistema de Gestión de Citas Médicas, verificando su relación con los requerimientos funcionales, la arquitectura por capas, la persistencia en SQL Server y los controles básicos de seguridad.

2.1 Objetivos específicos

- Identificar las tecnologías, proyectos y capas que conforman el software entregado.
- Describir los módulos de usuarios, pacientes, médicos, enfermería, especialidades, horarios, citas, diagnósticos, menús y permisos.
- Relacionar los casos de uso principales con las páginas y clases del proyecto.
- Definir medidas de seguridad y buenas prácticas para el manejo de usuarios y datos clínicos.
- Proponer pruebas funcionales y de integración para validar el comportamiento del sistema.

Descripción del software entregado

La aplicación está orientada a apoyar la administración de una clínica. En el proyecto se identifican páginas ASPX para iniciar sesión, mostrar el panel general y gestionar pacientes, usuarios, médicos, enfermeras, horarios, reservas y atención de citas. La solución utiliza C# y ASP.NET Web Forms sobre .NET Framework; los proyectos de capas están configurados para .NET Framework 4.7.2. El README del proyecto menciona Visual Studio 2019 y SQL Server Management Studio, además de bibliotecas de interfaz como jQuery, DatePicker y DataTable.

Elemento	Identificación en el proyecto	Responsabilidad
Aplicación web	PaginaWebClinina / CapaPresentacion.csproj	Páginas ASPX, navegación, formularios y eventos de interfaz.
Capa de entidades	CapaEntidades	Clases de dominio: Usuario, Paciente, Cita, Médico, Enfermera, Especialidad, Horario, Diagnóstico, entre otras.
Lógica de negocio	CapaLogicaNegocio	Clases LN que validan y coordinan operaciones antes de acceder a los datos.
Acceso a datos	CapaAccesoDatos	Clases DAO y conexión a SQL Server para consultar y modificar registros.
Base de datos	Archivo SENA(3).bacpac	Exportación de esquema y datos de SQL Server; debe importarse en una instancia de pruebas.

Requerimientos del sistema

4.1 Requerimientos funcionales

Código	Requerimiento	Módulo relacionado	Criterio de aceptación
RF-01	Permitir el inicio de sesión de usuarios registrados.	Login / Usuario	Con credenciales válidas permite ingresar; con datos inválidos muestra un mensaje controlado.
RF-02	Administrar usuarios y sus permisos.	GestionarUsuario / Permisos	El administrador puede consultar y gestionar usuarios según los permisos asignados.
RF-03	Registrar y consultar pacientes.	GestionarPaciente / PacienteDAO	Los datos se validan y quedan almacenados en SQL Server.
RF-04	Administrar médicos y especialidades.	GestionarMedico / Especialidad	Los médicos pueden asociarse a una especialidad válida.
RF-05	Administrar personal de enfermería.	GestionarEnfermera	Permite consultar y mantener la información de enfermería.
RF-06	Gestionar horarios de atención.	GestionarHorarioAtencion / horarios de enfermería	Se guardan horarios válidos y se evita asignar franjas incompatibles.
RF-07	Registrar y consultar reservas de citas.	GestionarReservaCitas / CitaDAO	La cita se relaciona con el paciente y los datos de atención requeridos.
RF-08	Gestionar la atención de la cita y diagnósticos.	GestionarAtencionCita / DiagnosticoDAO	La atención se registra vinculada a la cita correspondiente.
RF-09	Mostrar opciones de navegación según permisos.	Menús / Permisos	Cada perfil visualiza únicamente las opciones autorizadas.
RF-10	Mostrar mensajes de error controlados.	ErrorPage / BadRequest	Los errores no exponen credenciales, cadenas de conexión ni detalles internos.

4.2 Requerimientos no funcionales

- Seguridad: validar entradas, controlar sesiones y restringir funciones por permisos.
- Integridad: mantener relaciones coherentes entre pacientes, citas, médicos, especialidades y horarios.
- Usabilidad: formularios claros, mensajes comprensibles y navegación consistente.
- Mantenibilidad: separar presentación, entidades, lógica de negocio y acceso a datos.
- Compatibilidad: ejecutar la solución en un entorno compatible con ASP.NET Web Forms y .NET Framework 4.7.2.
- Privacidad: limitar el acceso a la información clínica y evitar exponer datos personales en mensajes o registros técnicos.

Arquitectura e integración por capas

La arquitectura observada corresponde a una separación por responsabilidades. Las páginas ASPX reciben las acciones del usuario; la lógica de negocio aplica reglas y coordina procesos; las entidades representan la información; y los DAO encapsulan operaciones de persistencia. La conexión con SQL Server se centraliza en la clase Conexion de la capa de acceso a datos.

Capa	Componentes observados	Flujo de integración
Presentación	login.aspx, PanelGeneral.aspx, GestionarPaciente.aspx, GestionarUsuario.aspx, GestionarMedico.aspx, GestionarEnfermera.aspx, GestionarReservaCitas.aspx, GestionarAtencionCita.aspx	Recibe datos del formulario, presenta resultados y llama a la lógica de negocio.
Lógica de negocio	UsuarioLN, PacienteLN, CitaLN, MedicoLN, EnfermeraLN, EspecialidadLN, HorarioAtencionLN, DiagnosticoLN, PermisoLN, MenuLN	Valida reglas del proceso y solicita operaciones a los DAO.
Entidades	Usuario, Paciente, Cita, Medico, Enfermera, Especialidad, HorarioAtencion, Diagnostico, Permiso, Menu	Transporta y organiza los datos del dominio.
Acceso a datos	UsuarioDAO, PacienteDAO, CitaDAO, MedicoDAO, EnfermeraDAO, EspecialidadDAO, HorarioAtencionDAO, DiagnosticoDAO, PermisoDAO, MenuDAO, Conexion	Ejecuta consultas y operaciones contra SQL Server.

Flujo general de una operación:

1. El usuario inicia sesión o abre una función desde la navegación.
2. La página de presentación valida los campos obligatorios y envía la solicitud a la clase LN correspondiente.
3. La capa de negocio comprueba las reglas y prepara la operación.
4. El DAO ejecuta la consulta parametrizada en SQL Server mediante la conexión configurada.
5. El resultado regresa a la capa de negocio y luego a la página, que muestra una confirmación o un mensaje de error.

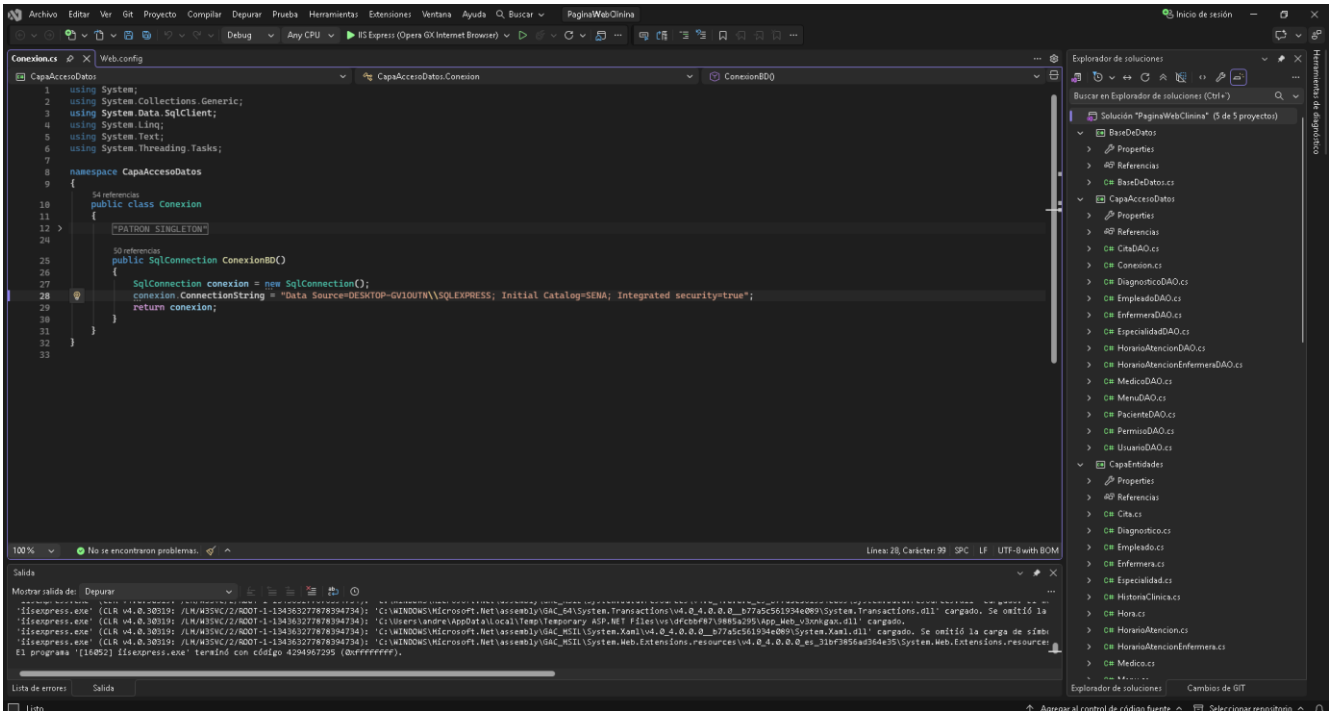
Módulos y componentes integrados

Módulo	Componentes asociados	Función dentro del sistema
Autenticación	login.aspx, login.aspx.cs, UsuarioLN, UsuarioDAO, Usuario	Identificar al usuario y permitir el acceso de acuerdo con el resultado de autenticación.
Usuarios y permisos	GestionarUsuario.aspx, UsuarioLN, UsuarioDAO, PermisoLN, PermisoDAO	Administrar cuentas y reglas de acceso.
Pacientes	GestionarPaciente.aspx, PacienteLN, PacienteDAO, Paciente	Registrar, consultar y mantener los datos del paciente.
Médicos y especialidades	GestionarMedico.aspx, MedicoLN, MedicoDAO, EspecialidadLN, EspecialidadDAO	Administrar médicos y la especialidad asociada.
Enfermería	GestionarEnfermera.aspx, EnfermeraLN, EnfermeraDAO	Administrar la información de enfermeras.
Horarios	GestionarHorarioAtencion.aspx, GestionarHorariosEnfermeras, HorarioAtencionLN y DAO relacionado	Gestionar disponibilidad y horarios de atención.

Reservas de citas	GestionarReservaCitas.aspx, CitaLN, CitaDAO, Cita	Crear y consultar citas vinculadas a pacientes y recursos de atención.
Atención y diagnóstico	GestionarAtencionCita.aspx, GestionarAtencionPaciente.aspx, DiagnosticoLN, DiagnosticoDAO	Registrar la atención y su información diagnóstica.
Menús	GestionarMenus.aspx, MenuLN, MenuDAO	Administrar opciones de navegación.
Errores	ErrorPage.aspx, BadRequest.aspx	Mostrar respuestas controladas ante errores o solicitudes inválidas.

Casos de uso principales

Actor	Caso de uso	Flujo principal	Resultado esperado
Usuario autorizado	Iniciar sesión	Ingresa usuario y contraseña; el sistema valida las credenciales.	Acceso concedido o mensaje de error genérico.
Administrador	Gestionar usuarios	Abre el módulo, consulta registros y realiza la operación disponible.	Datos actualizados y permisos respetados.
Personal de recepción	Registrar paciente	Diligencia el formulario, verifica campos y guarda.	Paciente persistido y visible en consulta.
Administrador / personal autorizado	Gestionar médico	Registra o consulta médico y especialidad.	Información del médico disponible para los procesos de citas.
Personal autorizado	Gestionar horarios	Selecciona recurso, día y franja horaria.	Horario válido guardado sin conflictos.
Personal de recepción	Reservar cita	Busca o selecciona paciente, revisa disponibilidad y registra la cita.	Cita relacionada con el paciente y horario correspondiente.
Personal clínico autorizado	Registrar atención	Localiza la cita y registra la atención/diagnóstico.	Información de atención asociada a la cita.



Nota: los actores y flujos anteriores describen el comportamiento esperado del sistema a partir de los módulos identificados. Los permisos concretos deben confirmarse con la configuración y las reglas implementadas en el entorno de ejecución.

Mapa de navegación propuesto a partir de las páginas

Inicio → Login → Panel general. Desde el panel, según los permisos del usuario, se accede a:

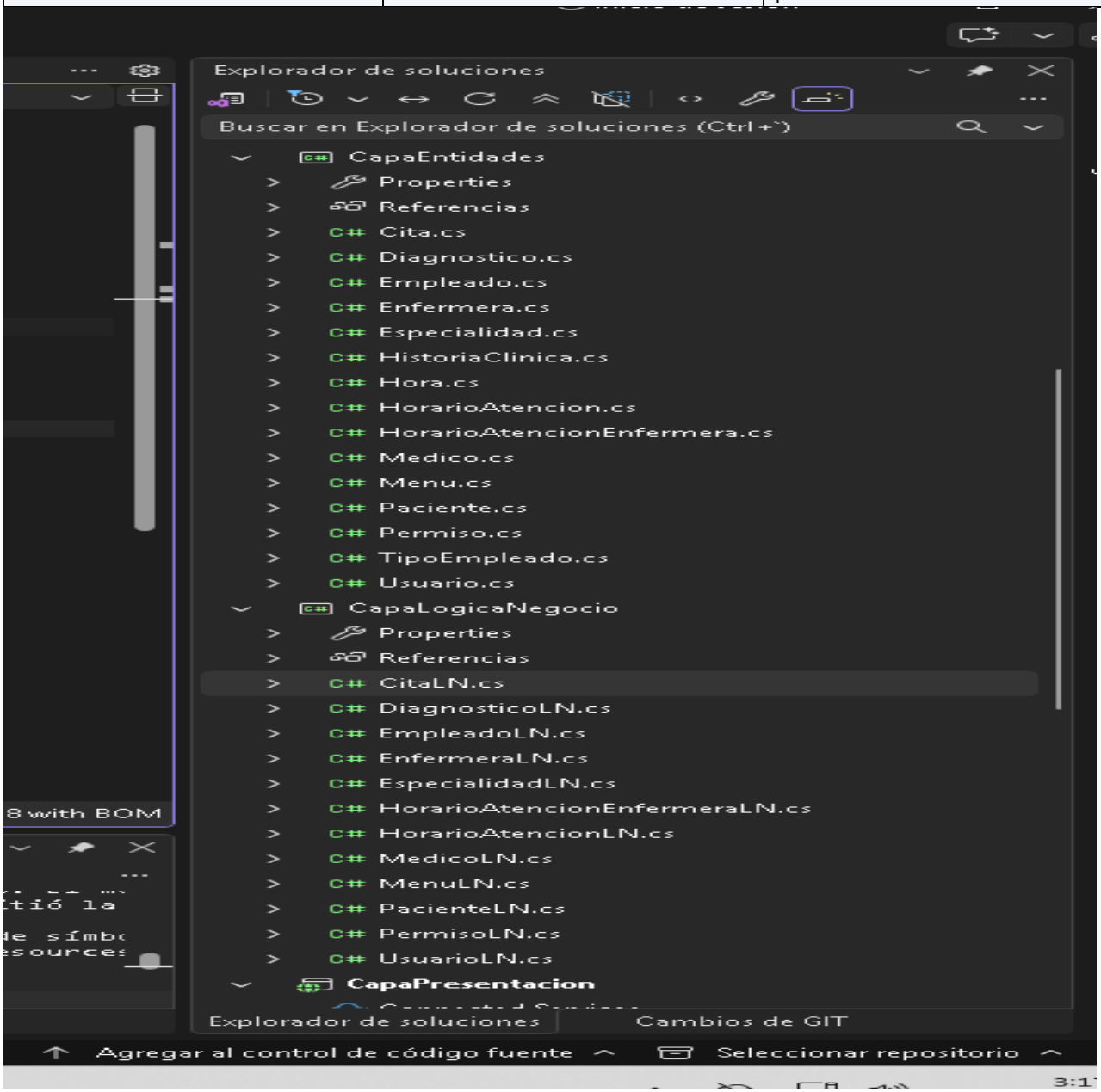
- Usuarios y permisos → Gestionar usuarios / gestionar permisos.
- Pacientes → Gestionar pacientes.
- Personal → Gestionar médicos / gestionar enfermeras.
- Agenda → Gestionar horarios / gestionar reservas de citas.
- Atención clínica → Gestionar atención de cita / atención de paciente / diagnósticos.
- Configuración de navegación → Gestionar menús.
- Errores y solicitudes inválidas → páginas de error controlado.

Este mapa es una representación funcional resumida, no una captura exacta del menú en ejecución. La navegación final depende de las opciones que cargue la aplicación y de los permisos asignados.

Modelo de datos y persistencia

Se entregó un archivo BACPAC, formato utilizado para transportar el esquema y los datos de una base de datos SQL Server. En su contenido se identifican, entre otras, tablas de usuario, permisos, menú, empleado, tipo de empleado, especialidad, hora y día de semana. Las entidades del código muestran además conceptos de paciente, cita, médico, enfermería, horario y diagnóstico. Antes de desplegar, se debe importar el BACPAC en una base de datos de pruebas y verificar que el esquema importado corresponda con las consultas utilizadas por los DAO.

Entidad o conjunto de datos	Propósito	Validación recomendada
Usuario / Permisos	Acceso y autorización.	Usuario único, contraseña protegida y permisos asignados de forma explícita.
Paciente	Información de identificación y contacto.	Documento no duplicado cuando aplique, campos obligatorios y validación de formato.
Empleado / Médico / Enfermera	Personal de la clínica.	Integridad de identificadores y relaciones con tipo de empleado o especialidad.
Especialidad	Clasificación de atención médica.	La especialidad asignada debe existir y estar activa.
Cita / Horario / Hora / DíaSemana	Programación y disponibilidad.	Fecha y hora válidas, disponibilidad y prevención de doble reserva.
Diagnóstico / Historia clínica	Información asociada a la atención.	Acceso restringido y asociación con la atención o cita correcta.
Menú	Opciones de navegación.	Las opciones visibles deben respetar los permisos del usuario.

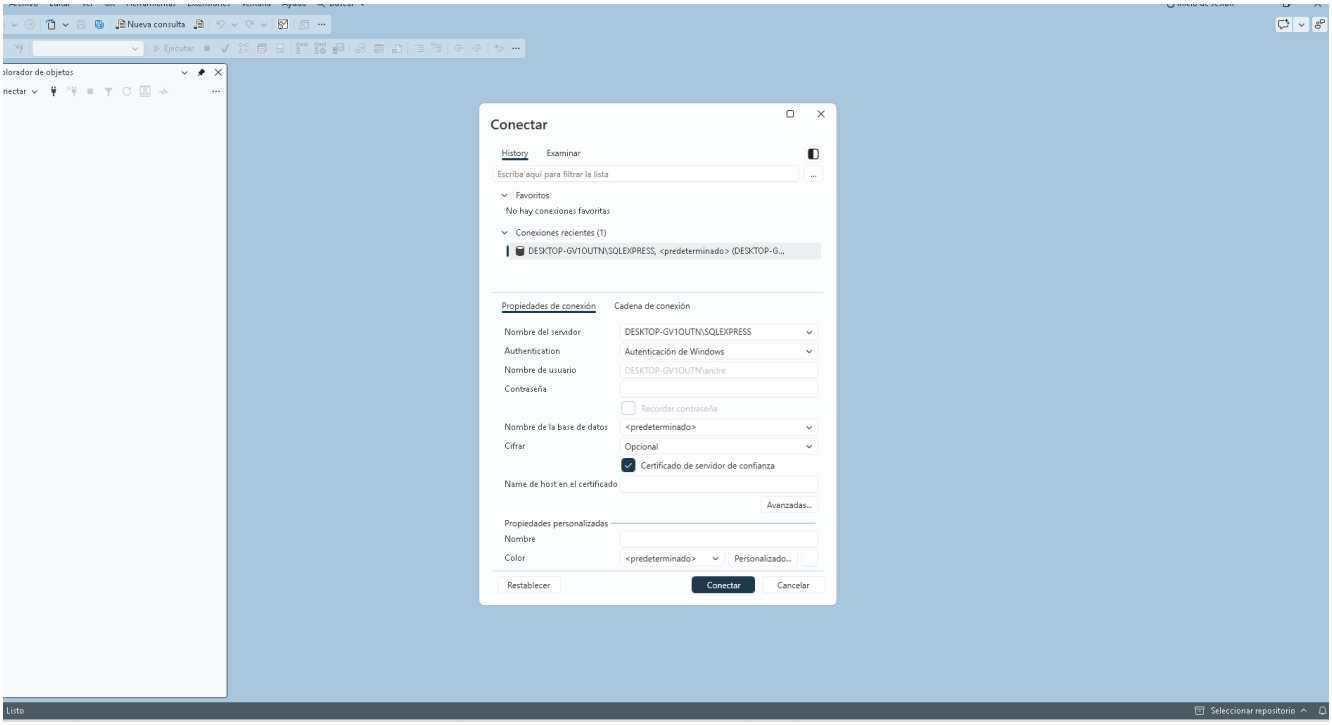


Seguridad de la aplicación

Para un sistema que administra datos personales y clínicos, se recomienda comprobar y aplicar como mínimo los siguientes controles:

- Autenticación: verificar las credenciales en el servidor y no confiar únicamente en validaciones del navegador.
- Contraseñas: almacenar hashes seguros con sal; nunca guardar contraseñas en texto plano ni mostrarlas en pantalla.
- Autorización: comprobar permisos en cada operación sensible del servidor, no solo ocultar botones o menús.
- SQL Injection: usar consultas parametrizadas o procedimientos almacenados con parámetros; evitar concatenar datos ingresados por usuarios en SQL.
- Sesiones: regenerar o establecer identificadores de sesión tras iniciar sesión, cerrar sesión correctamente y configurar tiempos de expiración.
- Validación: validar tipo, longitud, rango y obligatoriedad de cada dato tanto en presentación como en servidor.
- Errores: no mostrar trazas, rutas internas, cadenas de conexión ni consultas SQL al usuario final.
- Conexiones: mantener credenciales de base de datos fuera del código fuente y proteger los archivos de configuración.
- Transporte: desplegar con HTTPS en ambientes accesibles por red.
- Privacidad: limitar el acceso a historias clínicas y diagnósticos por rol; no registrar datos médicos innecesarios en logs.
- Protección web: revisar el uso de cookies seguras, protección CSRF en formularios sensibles y codificación de salida para reducir XSS.
- Copias de seguridad: establecer copias de respaldo, controlar el acceso y probar la restauración periódicamente.

Importante: estas son medidas recomendadas para validar o reforzar. La existencia de una página o clase con un nombre relacionado con seguridad no demuestra por sí sola que todos los controles estén implementados.



Buenas prácticas de codificación

- Conservar la separación entre presentación, lógica de negocio, entidades y acceso a datos.
- Mantener nombres de clases y métodos consistentes y descriptivos.
- Evitar duplicar reglas de validación entre páginas; centralizar reglas críticas en la capa de negocio.
- Liberar recursos de base de datos correctamente mediante using y manejo controlado de excepciones.
- No exponer excepciones técnicas al usuario; registrar detalles técnicos sin incluir contraseñas ni datos clínicos sensibles.
- Usar control de versiones Git para registrar cambios, mantener una rama estable y poder revertir errores.
- Separar la configuración de desarrollo, pruebas y producción.
- Revisar dependencias antiguas y actualizar únicamente después de validar compatibilidad.

Entorno de desarrollo y despliegue

Elemento	Configuración identificada o recomendada
IDE	Visual Studio; el README indica Visual Studio 2019.
Lenguaje y plataforma	C# / ASP.NET Web Forms / .NET Framework 4.7.2 en los proyectos de capas.
Base de datos	Microsoft SQL Server; administración con SQL Server Management Studio.
Interfaz	ASPX, CSS y bibliotecas mencionadas en el README: jQuery, DatePicker y DataTable.

Servidor web	IIS o IIS Express compatible con ASP.NET y .NET Framework.
Base de datos para pruebas	Importar SENA(3).bacpac en una instancia de pruebas; comprobar tablas, relaciones y datos.
Configuración	Revisar Web.config y cadena de conexión antes de ejecutar; usar credenciales de prueba y no publicar secretos.
Control de versiones	Git recomendado; excluir binarios, archivos temporales, secretos y carpetas de IDE innecesarias.

12.1 Procedimiento recomendado para preparar el ambiente

6. Abrir PaginaWebClinina.sln en Visual Studio.
7. Verificar que las referencias entre CapaPresentacion, CapaLogicaNegocio, CapaEntidades y CapaAccesoDatos estén resueltas.
8. Restaurar paquetes NuGet si el IDE lo solicita y compilar la solución.
9. Importar el archivo BACPAC en SQL Server Management Studio con un nombre de base de datos de pruebas.
10. Revisar la cadena de conexión configurada en Web.config y adaptarla al servidor de pruebas.
11. Ejecutar la aplicación con IIS Express; probar primero login, navegación y operaciones básicas de cada módulo.
12. Documentar errores encontrados y repetir las pruebas después de cada corrección.

Plan de pruebas unitarias, funcionales y de integración

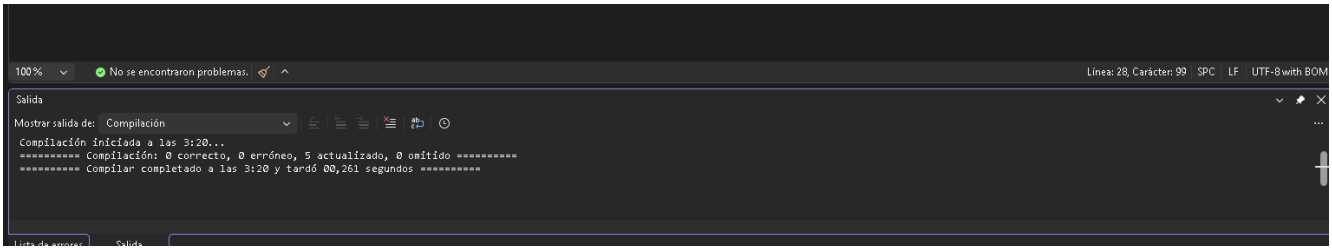
Las siguientes pruebas se proponen para verificar los módulos. Debe registrarse el resultado real durante su ejecución en el entorno configurado. No se marcan como aprobadas porque no se ha ejecutado la aplicación desde este documento.

ID	Prueba	Procedimiento	Resultado esperado	Estado
P-01	Compilación	Compilar la solución en Visual Studio.	Sin errores de compilación ni referencias faltantes.	Pendiente
P-02	Login válido	Ingresar credenciales de una cuenta de pruebas.	Acceso correcto y navegación al panel autorizado.	Pendiente
P-03	Login inválido	Ingresar contraseña incorrecta.	Acceso denegado con mensaje genérico.	Pendiente
P-04	Acceso por permisos	Intentar abrir una función sin permiso.	El servidor deniega la operación.	Pendiente
P-05	Registro de paciente	Completar campos válidos y guardar.	Registro persistido y visible al consultar.	Pendiente
P-06	Validación de paciente	Enviar campos obligatorios vacíos o formatos incorrectos.	El sistema informa la validación y no guarda datos inválidos.	Pendiente
P-07	Gestión de médico	Registrar/consultar médico con especialidad existente.	Relación válida y datos consultables.	Pendiente
P-08	Reserva de cita	Crear una cita en un horario disponible.	Cita guardada con relaciones correctas.	Pendiente
P-09	Conflicto de horario	Intentar reservar la misma franja ocupada.	El sistema evita la doble reserva o informa el conflicto.	Pendiente

P-10	Atención y diagnóstico	Registrar atención asociada a una cita existente.	Datos guardados y asociados correctamente.	Pendiente
P-11	Persistencia	Guardar un registro y consultarlo de nuevo.	Los datos permanecen en SQL Server.	Pendiente
P-12	Errores controlados	Provocar una entrada inválida o error controlado.	No se exponen datos técnicos sensibles.	Pendiente

13.1 Evidencia de ejecución que se debe anexar

- Captura de la solución compilada sin errores.
- Capturas de login válido e inválido usando cuentas de prueba.
- Capturas de los módulos de pacientes, usuarios, médicos y reservas.
- Consulta en SQL Server que demuestre la persistencia de un registro de prueba, ocultando datos personales reales.
- Tabla de resultados con fecha, responsable, resultado obtenido y observaciones.
- Registro de los defectos encontrados y las correcciones realizadas.



Control de versiones

Se recomienda administrar el proyecto mediante Git para mantener un historial de cambios y facilitar el trabajo por módulos. Cada cambio debe describir su objetivo y asociarse con una prueba. Ejemplo de flujo de trabajo:

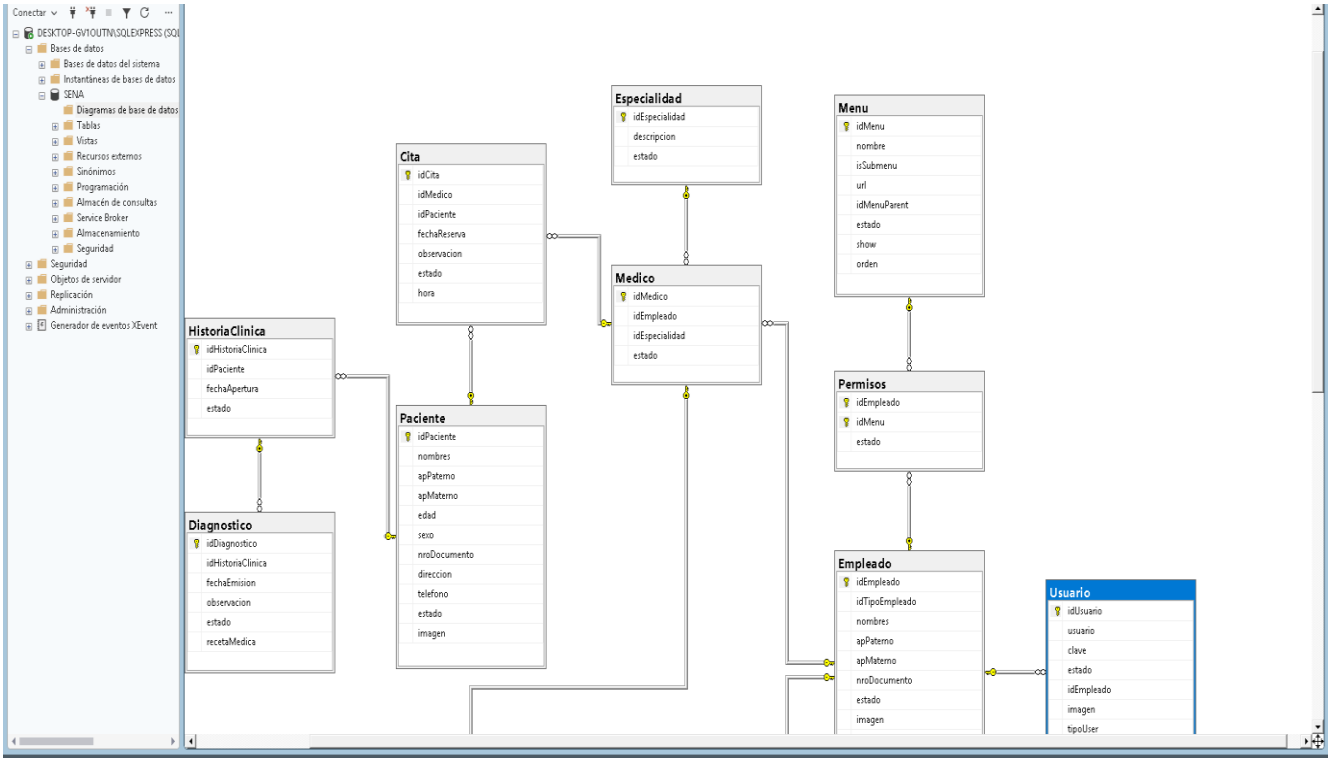
13. Crear una rama por funcionalidad o corrección (por ejemplo, feature/registro-paciente).
14. Realizar cambios pequeños y coherentes.
15. Compilar y ejecutar las pruebas correspondientes antes de integrar.
16. Registrar cambios con mensajes descriptivos.
17. Revisar diferencias y fusionar en la rama estable únicamente después de validar.

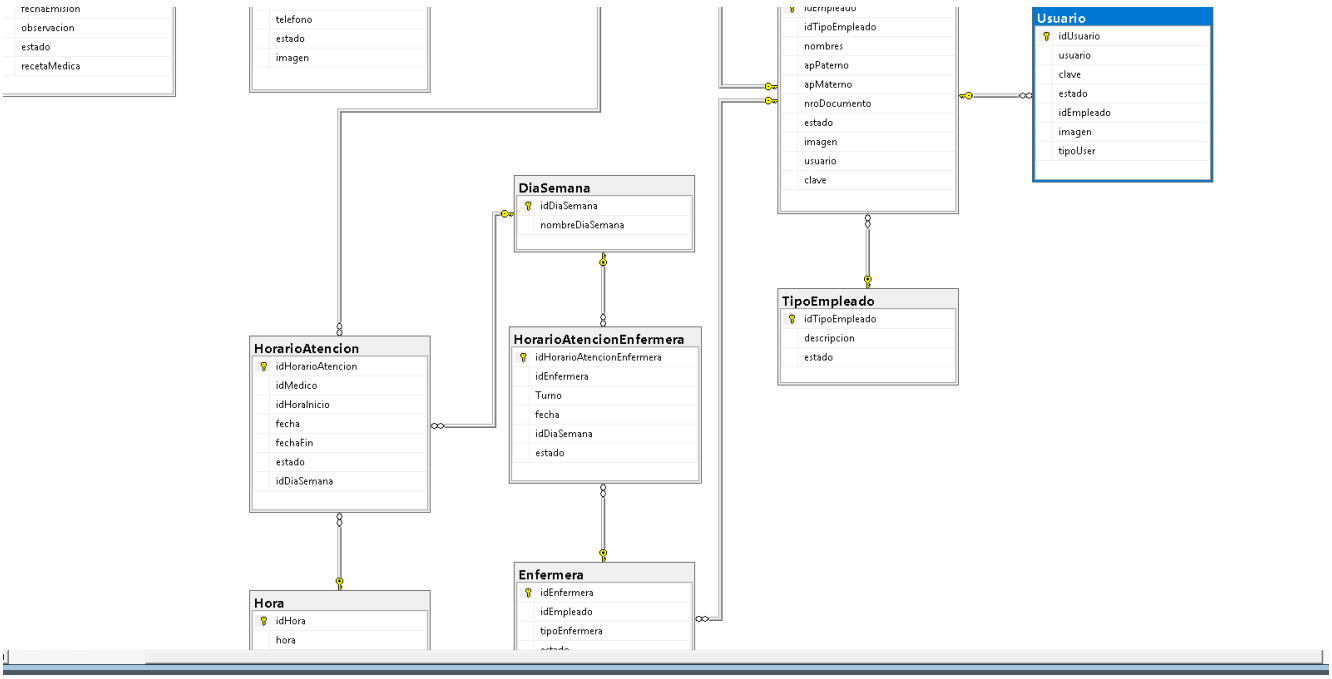
Ejemplos de comandos:

```
git init
git add .
git commit -m "Documenta e integra módulos del sistema de citas"
git status
```

15. Diagrama lógico de componentes

Representación textual de la integración por capas:


Conectar
DESKTOP-GVTOUTN\SQLEXPRESS (SQL
Bases de datos
Bases de datos del sistema
Instantáneas de bases de datos
SENA
Diagramas de base de datos
Tablas
Vistas
Recursos externos
Sinónimos
Programación
Almacén de consultas
Service Broker
Almacenamiento
Seguridad
Seguridad
Objetos de servidor
Replicación
Administración
Generador de eventos XEvent
Especialidad
idEspecialidad
descripcion
estado
Menu
idMenu
nombre
isSubmenu
url
idMenuParent
estado
show
orden
Cita
idCita
idMedico
idPaciente
fechaReserva
observacion
estado
hora
Medico
idMedico
idEmpleado
idEspecialidad
estado
HistoriaClinica
idHistoriaClinica
idPaciente
fechaApertura
estado
Paciente
idPaciente
nombres
apPaterno
apMaterno
edad
sexo
nroDocumento
direccion
telefono
estado
imagen
Permisos
idEmpleado
idMenu
estado
Diagnostico
idDiagnostico
idHistoriaClinica
fechaEmision
observacion
estado
recetaMedica
Empleado
idEmpleado
idTipoEmpleado
nombres
apPaterno
apMaterno
nroDocumento
estado
imagen
Usuario
idUsuario
usuario
clave
estado
idEmpleado
imagen
tipoUser


fechaEmision
observacion
estado
recetaMedica
telefono
estado
imagen
idEmpleado
idTipoEmpleado
nombres
apPaterno
apMaterno
nroDocumento
estado
imagen
usuario
clave
Usuario
idUsuario
usuario
clave
estado
idEmpleado
imagen
tipoUser
DiaSemana
idDiaSemana
nombreDiaSemana
TipoEmpleado
idTipoEmpleado
descripcion
estado
HorarioAtencion
idHorarioAtencion
idMedico
idHoraInicio
fecha
fechaFin
estado
idDiaSemana
HorarioAtencionEnfermera
idHorarioAtencionEnfermera
idEnfermera
Turno
fecha
idDiaSemana
estado
Enfermera
idEnfermera
idEmpleado
tipoEnfermera
Hora
idHora
hora

[Usuario]
↓
[Presentación: páginas ASPX / code-behind]
↓
[Lógica de negocio: clases LN]
↓
[Entidades del dominio]
↕
[Acceso a datos: DAO + Conexion]
↕
[SQL Server: usuarios, pacientes, citas, horarios, personal, permisos]

Resultados y conclusión

